

SIMATS ENGINEERING

ASSESSMENT TOOL 3

COURSE NAME: Database Management

Systems COURSE CODE: CSA05

COURSE OUTCOME COVERED

CO5: *Implement database operations using query processing, optimization, and transaction management to ensure consistency and reliability. (BL3, BL4)*

Activity Tool : Mock Interview

(Low) Assessment Tool Weightage:

30%

QUESTIONS		Total Marks: 50	
Q.No	Question	Bloom's Level	Marks

1	Interviewer: 'Explain the steps of query processing in DBMS. How does a query optimizer decide the best execution plan?'	BL3 – Apply	5
2	Interviewer: 'What is the difference between heuristic optimization and cost-based optimization? When would you use each?'	BL4 – Analyze	5
3	Interviewer: 'How would you implement database connectivity in a real project? Explain the difference between JDBC and ODBC.'	BL3 – Apply	5
4	Interviewer: 'What are ACID properties? Give a real-world example where each property is critical.'	BL3 – Apply	5
5	Interviewer: 'Explain concurrency control problems: dirty read, lost update, and unrepeatable read. How does 2PL solve them?'	BL4 – Analyze	5
6	Interviewer: 'What is deadlock in DBMS? Describe the deadlock detection algorithm and at least two prevention strategies.'	BL4 – Analyze	5
7	Interviewer: 'Compare Immediate Update and Deferred Update recovery. In what scenarios is each preferred?'	BL4 – Analyze	5
8	Interviewer: 'Explain Shadow Paging. What are its advantages and limitations compared to log-based recovery?'	BL3 – Apply	5
9	Interviewer: 'What is a serializability schedule? How do you test for conflict serializability using a precedence graph?'	BL4 – Analyze	5
10	Interviewer: 'Describe the Two-Phase Locking protocol variations: Basic 2PL, Strict 2PL, and Rigorous 2PL. Which is most commonly used and why?'	BL4 – Analyze	5

Code of Conduct:

*I **ADHAVAN CHANDRASEKAR(192521209)** certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: ADHAVAN CHANDRASEKAR(192521209)

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Technical Knowledge	25	Demonstrates strong and accurate subject knowledge with in-depth understanding	Shows good knowledge with minor gaps	Basic knowledge with noticeable errors	Lacks fundamental technical knowledge
Problem Solving	25	Solves problems logically and applies concepts effectively in real scenarios	Applies concepts with moderate accuracy	Limited problem-solving ability; requires guidance	Unable to solve or apply concepts
Analytical Thinking	15	Analyzes questions critically and provides well-structured, logical answers	Provides reasonable analysis with some structure	Superficial or partially structured responses	No analytical thinking demonstrated
Communication	15	Communicates clearly, confidently, and professionally with proper terminology	Communicates well with minor hesitation	Communication lacks clarity or confidence	Unable to communicate effectively
Confidence	15	Displays high confidence, positive attitude, and professional behavior throughout	Shows good confidence with minor issues	Low confidence; inconsistent behavior	Lacks confidence and professionalism
Response to Questions	10	Responds accurately, promptly, and elaborates with relevant examples	Responds correctly but with limited depth	Partial responses; needs prompting	Unable to respond appropriately

Q1. EXPLAIN THE STEPS OF QUERY PROCESSING IN DBMS. HOW DOES A QUERY OPTIMIZER DECIDE THE BEST EXECUTION PLAN?

ANSWER

Query processing is the process of converting an SQL query into an efficient execution plan and producing the required result.

The main steps are:

1. Parsing and Validation:

The DBMS first checks the SQL query for syntax and semantic correctness. It verifies whether the tables, columns, operators, and data types are valid.

2. Query Translation:

The valid SQL query is converted into an internal representation such as a query tree or relational algebra expression.

3. Query Optimization:

The optimizer generates different possible execution plans. For example, it may choose different join orders, indexes, or join algorithms.

4. Cost Estimation:

Each plan is evaluated using estimated factors such as disk I/O, CPU processing, number of tuples, relation size, and available indexes.

5. Plan Selection:

The optimizer selects the plan having the lowest estimated cost.

6. Query Execution:

The selected execution plan is passed to the execution engine, which accesses the required data and performs the operations.

7. Result Generation:

The final result is returned to the user.

EXAMPLE

For example, if I have to join three large tables, the optimizer may compare different join orders. If one ordering creates a much smaller intermediate table, it may require fewer disk accesses and therefore be selected.

Q2. WHAT IS THE DIFFERENCE BETWEEN HEURISTIC OPTIMIZATION AND COST-BASED OPTIMIZATION? WHEN WOULD YOU USE EACH?

ANSWER

Both heuristic and cost-based optimization are techniques used to improve query execution, but they make decisions differently.

HEURISTIC OPTIMIZATION

Heuristic optimization follows predefined rules or strategies instead of calculating the exact cost of every possible plan.

Common heuristics include:

- Perform selection operations early.
- Perform projection early.
- Reduce the size of intermediate relations.
- Perform highly selective operations first.
- Choose appropriate join ordering.

For example, if a query selects only 10 records from a table containing one million records, applying the selection first can greatly reduce the amount of data processed by subsequent operations.

COST-BASED OPTIMIZATION

Cost-based optimization generates multiple possible execution plans and estimates their costs using factors such as:

- Disk I/O
- CPU usage
- Relation size
- Number of tuples
- Available indexes
- Join algorithms
- Intermediate result sizes

The plan with the lowest estimated cost is selected.

COMPARISON

Heuristic Optimization	Cost-Based Optimization
Uses predefined rules	Uses estimated execution costs
Faster optimization	More computation during optimization
Does not require detailed cost estimation	Requires statistics and cost estimation
Suitable for simpler optimization	Suitable for complex queries
May not always find the best plan	Usually selects a better plan

WHEN WOULD I USE EACH?

I would use **heuristic optimization** when quick optimization is required and the query is relatively simple.

I would use **cost-based optimization** for complex queries involving multiple joins, large tables, and different possible execution plans, because it can make a more informed decision.

The main difference is that **heuristic optimization relies on rules, while cost-based optimization relies on estimated execution cost.**

Q3. HOW WOULD YOU IMPLEMENT DATABASE CONNECTIVITY IN A REAL PROJECT? EXPLAIN THE DIFFERENCE BETWEEN JDBC AND ODBC.

ANSWER

In a real project, I would implement database connectivity through a structured connection-management process.

The basic steps would be:

1. Configure the database URL, username, and password.
2. Load or configure the required database driver.
3. Establish a database connection.
4. Create a statement or prepared statement.
5. Execute the SQL query.
6. Process the returned result.
7. Commit or rollback the transaction when required.
8. Close or return the connection to the connection pool.

For a large application, I would use **connection pooling** so that existing connections can be reused instead of creating a new connection for every query.

JDBC

JDBC stands for **Java Database Connectivity**.

It is a Java-based API that allows Java applications to communicate with databases through JDBC drivers.

ODBC

ODBC stands for **Open Database Connectivity**.

It is a general database connectivity standard that allows applications to communicate with different database systems using ODBC drivers.

COMPARISON

JDBC	ODBC
Designed mainly for Java applications	Language-independent standard
Java-based API	General database API
Uses JDBC drivers	Uses ODBC drivers
Commonly used in Java applications	Can be used by applications in different programming environments

REAL-WORLD EXAMPLE

In an online banking application developed in Java, I could use JDBC to connect the application to the database, retrieve account information, update transactions, and commit or rollback transactions safely.

Q4. WHAT ARE ACID PROPERTIES? GIVE A REAL-WORLD EXAMPLE WHERE EACH PROPERTY IS CRITICAL.

ANSWER

ACID properties ensure that database transactions are processed reliably and maintain database consistency.

ACID stands for:

A – Atomicity

C – Consistency

I – Isolation

D – Durability

1. ATOMICITY

Atomicity means that a transaction is treated as a single unit. Either all operations are completed or none of them are applied.

Example:

In a bank transfer of ₹5,000 from Account A to Account B, money should not be deducted from A unless it is successfully credited to B. If the transaction fails, the entire operation should be rolled back.

2. CONSISTENCY

Consistency ensures that a transaction changes the database from one valid state to another valid state while maintaining all constraints.

Example:

If a banking rule does not allow an account balance to become negative, a transaction should not violate this constraint.

3. ISOLATION

Isolation ensures that concurrently executing transactions do not interfere with each other in an incorrect manner.

Example:

If two users try to purchase the last available product simultaneously, the database must prevent both transactions from incorrectly believing that the product is available.

4. DURABILITY

Durability means that once a transaction is committed, its changes remain permanently stored even if a system failure occurs.

Example:

After a bank transfer is successfully committed, the updated account balances must remain correct even if the database server crashes immediately afterward.

Q5. EXPLAIN CONCURRENCY CONTROL PROBLEMS: DIRTY READ, LOST UPDATE, AND UNREPEATABLE READ. HOW DOES 2PL SOLVE THEM?

ANSWER

Concurrency control is used to manage multiple transactions executing at the same time while maintaining database consistency.

1. DIRTY READ

A dirty read occurs when one transaction reads data written by another transaction before that transaction commits.

For example:

T1: Update Balance = ₹5000

T2: Read Balance = ₹5000

T1: Rollback

T2 has read a value that never became permanent.

2. LOST UPDATE

A lost update occurs when two transactions read the same data and then update it, causing one update to overwrite the other.

For example:

T1: Read Balance = ₹1000

T2: Read Balance = ₹1000

T1: Write ₹1200

T2: Write ₹1100

The update made by T1 is lost.

3. UNREPEATABLE READ

An unrepeatable read occurs when a transaction reads the same row twice but obtains different values because another transaction modified and committed the row between the two reads.

HOW 2PL HELPS

Two-Phase Locking uses two phases:

Growing Phase:

Transactions acquire locks but do not release them.

Shrinking Phase:

Transactions release locks but do not acquire new locks.

Depending on the variant, locks may be held until commit.

By controlling when transactions can read or modify data, 2PL prevents conflicting operations from executing incorrectly and produces **conflict-serializable schedules**.

Q6. WHAT IS DEADLOCK IN DBMS? DESCRIBE THE DEADLOCK DETECTION ALGORITHM AND AT LEAST TWO PREVENTION STRATEGIES.

ANSWER

A deadlock occurs when two or more transactions wait indefinitely for resources held by each other.

For example:

T1 holds Lock A $\rightarrow$ waits for Lock B

T2 holds Lock B $\rightarrow$ waits for Lock A

Neither transaction can proceed.

DEADLOCK DETECTION

A common method is the **Wait-For Graph**.

In a Wait-For Graph:

- Each transaction is represented as a node.
- An edge $T_i \rightarrow T_j$ means T_i is waiting for a resource held by T_j .
- A cycle indicates a deadlock.

ALGORITHM

1. Construct the Wait-For Graph.
2. Represent every transaction as a node.
3. Add an edge when one transaction waits for another.
4. Perform DFS or another cycle-detection method.
5. If a cycle is found, report a deadlock.
6. Select a transaction as a victim.
7. Roll back the victim transaction to release its resources.

PREVENTION STRATEGY 1 – WAIT-DIE

Wait-Die uses transaction timestamps.

- An older transaction can wait for a younger transaction.
- A younger transaction requesting a resource held by an older transaction is aborted.

This prevents circular waiting.

PREVENTION STRATEGY 2 – WOUND-WAIT

Wound-Wait also uses timestamps.

- An older transaction can force a younger transaction to roll back.
- A younger transaction waits when the resource is held by an older transaction.

Q7. COMPARE IMMEDIATE UPDATE AND DEFERRED UPDATE RECOVERY. IN WHAT SCENARIOS IS EACH PREFERRED?

ANSWER

Immediate Update and Deferred Update differ mainly in **when database changes are written to the actual database.**

IMMEDIATE UPDATE

In Immediate Update, changes can be written to the database before the transaction commits.

Therefore, after a crash:

- Committed transactions may need **REDO**.
- Uncommitted transactions may need **UNDO**.

DEFERRED UPDATE

In Deferred Update, modifications are not written to the database until the transaction commits.

Therefore, if a crash occurs:

- Uncommitted changes do not need to be undone from the database.
- Committed operations may need to be **REDO**.

COMPARISON

Immediate Update	Deferred Update
Changes can reach disk before commit	Changes are delayed until commit
May require UNDO and REDO	Mainly requires REDO
More flexible buffer management	Simpler recovery
Recovery is more complex	Recovery is relatively simpler
Useful for systems requiring efficient immediate writes	Useful when simpler recovery is preferred

WHEN WOULD I USE EACH?

I would prefer **Immediate Update** when the system requires flexible buffer management and efficient handling of frequent database updates.

I would prefer **Deferred Update** when simpler recovery logic is more important and delaying database writes until commit is acceptable.

Q8. EXPLAIN SHADOW PAGING. WHAT ARE ITS ADVANTAGES AND LIMITATIONS COMPARED TO LOG-BASED RECOVERY?

ANSWER

Shadow Paging is a recovery technique that maintains two page tables:

- **Shadow Page Table**
- **Current Page Table**

The shadow page table points to the stable version of the database, while the current page table represents the updated version.

When a transaction modifies a page, the original page is not overwritten. Instead, a new page is allocated and the current page table is updated to point to it.

WORKING

Transaction Starts



Create Shadow Page Table



Create Current Page Table



Perform Updates



New Pages Are Allocated



Commit

Failure



Current Table



Shadow Table

Becomes Active

Restored

If the transaction commits, the current page table becomes the new valid page table.

If a failure occurs, the system uses the shadow page table to restore the previous consistent state.

ADVANTAGES

1. No traditional UNDO operation is required.
2. Recovery can be relatively fast.
3. The original database pages remain protected.
4. It provides a simple rollback mechanism.

LIMITATIONS

1. Maintaining two page tables creates additional overhead.
2. Large databases can require significant page-table management.
3. Fragmentation can occur because new pages are repeatedly allocated.
4. It is less flexible for systems with frequent concurrent transactions.
5. Log-based systems such as ARIES are generally more suitable for complex transaction processing.

Q9. WHAT IS A SERIALIZABILITY SCHEDULE? HOW DO YOU TEST FOR CONFLICT SERIALIZABILITY USING A PRECEDENCE GRAPH?

ANSWER

A serializability schedule is a concurrent schedule whose result is equivalent to executing the same transactions serially in some order.

A **serial schedule** executes one transaction completely before starting another transaction.

A **serializable schedule** can interleave operations but still produces a result equivalent to some serial execution.

CONFLICT SERIALIZABILITY

Two operations conflict when:

1. They belong to different transactions.
2. They operate on the same data item.
3. At least one of them is a write operation.

Examples are:

Read(X) – Write(X)

Write(X) – Read(X)

Write(X) – Write(X)

PRECEDENCE GRAPH

To test conflict serializability:

1. Create one node for every transaction.
2. Examine the schedule for conflicting operations.
3. If an operation of T_i occurs before a conflicting operation of T_j , create an edge:

$T_i \rightarrow T_j$

4. Continue until all conflicts are represented.
5. Check the graph for cycles.

ANALYSIS

If the precedence graph is **acyclic**, the schedule is conflict serializable.

If the graph contains a **cycle**, the schedule is not conflict serializable.

EXAMPLE

Suppose:

T1: Read(X)

T1: Write(X)

T2: Read(X)

T2: Write(X)

Since T1's conflicting operations occur before T2's operations, we get:

$T1 \rightarrow T2$

There is no cycle, so the schedule is conflict serializable.

Q10. DESCRIBE THE TWO-PHASE LOCKING PROTOCOL VARIATIONS: BASIC 2PL, STRICT 2PL, AND RIGOROUS 2PL. WHICH IS MOST COMMONLY USED AND WHY?

ANSWER

Two-Phase Locking ensures serializability by dividing transaction execution into two phases:

Growing Phase: Locks are acquired.

Shrinking Phase: Locks are released.

There are three important variations.

1. BASIC 2PL

In Basic 2PL:

- A transaction can acquire locks during the growing phase.
- Once it releases a lock, it cannot acquire any new lock.
- Locks can be released before the transaction commits.

It provides conflict serializability but may allow cascading rollbacks.

2. STRICT 2PL

In Strict 2PL:

- The transaction follows the two-phase locking rule.
- Exclusive locks are held until the transaction commits or aborts.

This prevents other transactions from reading or overwriting uncommitted data.

3. RIGOROUS 2PL

In Rigorous 2PL:

- Both shared and exclusive locks are held until the transaction commits or aborts.

Therefore, no lock is released before the transaction finishes.

COMPARISON

Basic 2PL	Strict 2PL	Rigorous 2PL
Locks may be released before commit	Exclusive locks held until commit	All locks held until commit
Provides serializability	Provides serializability	Provides serializability
Cascading rollback may occur	Prevents cascading rollback	Prevents cascading rollback
More flexible	Stronger control	Strongest locking discipline

WHICH IS MOST COMMONLY USED?

Strict 2PL is commonly preferred in practical database systems because it provides a good balance between concurrency and recovery safety.

By holding exclusive locks until commit or abort, it prevents other transactions from reading or overwriting uncommitted data. This makes recovery simpler and avoids cascading rollbacks while still allowing shared locks to be released according to the protocol.

For practical transaction processing, **Strict 2PL is generally the preferred choice because it provides strong consistency and simplifies recovery without being as restrictive as Rigorous 2PL.**